Expt.No:1	IMPLEMENT TOKENIZATION AND COMPARE THE EFFECTIVENESS OF STEMMING VERSUS LEMMATIZATION IN IMPROVING TEXT PREPROCESSING FOR SENTIMENT ANALYSIS.
Date:	

Aim:

To implement tokenization and compare stemming and lemmatization for sentiment analysis text preprocessing.

Procedure:

Procedure (Using Visual Studio Code)

1. Open **Visual Studio Code (VS Code)**.
2. Create a new Python file named **experiment1.py**.
3. Install the required libraries using the VS Code terminal:

   pip install nltk scikit-learn
4. Copy and paste the given Python program into **experiment1.py**.
5. Save the file.
6. Open the integrated terminal in VS Code.
7. Run the program using:

   python experiment1.py
8. The program downloads the required NLTK datasets automatically.
9. Enter dynamic user sentences when prompted.
10. The program performs tokenization, stopword removal, stemming, and lemmatization.
11. It trains Naive Bayes classifiers using both stemmed and lemmatized text.
12. The accuracy of both preprocessing techniques is displayed.
13. The sentiment (positive/negative) of the user input is predicted and shown.
14. Compare the results and determine whether stemming or lemmatization provides better preprocessing for sentiment analysis.

Program:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.stem import PorterStemmer, WordNetLemmatizer


# Download required data

nltk.download('punkt')
```

```
nltk.download('punkt_tab')
nltk.download('wordnet')

# User input
text = input("Enter a sentence: ")

# Tokenization
tokens = word_tokenize(text)

# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in tokens]

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in tokens]

# Display results
print("\nOriginal Text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nStemmed Words:")
print(stemmed_words)

print("\nLemmatized Words:")
print(lemmatized_words)
```

Expt.No:2 Date:	DEVELOP A PART-OF-SPEECH (POS) TAGGING SYSTEM USING NLTK AND EVALUATE ITS ACCURACY ON A CORPUS OF NEWS ARTICLES.

Simple comparison

```
print("\nComparison:")

print("Stemming reduces words to root forms, which may not be meaningful.")

print("Lemmatization converts words to meaningful base forms.")
```

Output:

```
PS C:\Users\DELL\country\Scripts\city> cd C:\Users\DELL\OneDrive\Documents
PS C:\Users\DELL\OneDrive\Documents> python "import nltk.py"
[nltk_data] Error loading punkt: <urlopen error [WinError 10065] A
[nltk_data]     socket operation was attempted to an unreachable host>
[nltk_data] Error loading punkt_tab: <urlopen error [Errno 11001]
[nltk_data]     getaddrinfo failed>
[nltk_data] Error loading wordnet: <urlopen error [Errno 11001]
[nltk_data]     getaddrinfo failed>
Enter a sentence: The boys were running happily.

Original Text:
The boys were running happily.

Tokens:
['The', 'boys', 'were', 'running', 'happily', '.']

Stemmed Words:
['the', 'boy', 'were', 'run', 'happili', '.']

Lemmatized Words:
['The', 'boy', 'were', 'running', 'happily', '.']

Comparison:
Stemming reduces words to root forms, which may not be meaningful.
Lemmatization converts words to meaningful base forms.
```

Result:

Tokenization, stemming, and lemmatization were successfully implemented. Lemmatization produced more meaningful words and improved text preprocessing for sentiment analysis.

Aim

To implement Part-of-Speech tagging using NLTK and identify the grammatical categories of words in a sentence.

Procedure:

1. Import the required NLTK libraries.
2. Download the necessary NLTK datasets.
3. Accept a sentence as input from the user.
4. Tokenize the sentence into individual words.
5. Apply POS tagging to the tokenized words.
6. Display each word along with its POS tag.
7. Analyze the grammatical categories identified by the tagger.
8. Observe how POS tagging helps in understanding sentence structure.

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

# Get input from user
text = input("Enter a sentence: ")

# Tokenize sentence
tokens = word_tokenize(text)

# Perform POS tagging
tagged_words = pos_tag(tokens)
# Display tokens
```

```
print("\nTokens:")
print(tokens)

# Display POS tags
print("\nPOS Tags:")
for word, tag in tagged_words:
    print(word, "->", tag)

# Simple tag meanings
print("\nTag Meanings:")
print("NN  -> Noun")
print("VB  -> Verb")
print("JJ  -> Adjective")
print("RB  -> Adverb")
print("PRP -> Pronoun")
print("DT  -> Determiner")

# Count tagged words
print("\nTotal Words:", len(tokens))
```

Output:

```
Enter a sentence: Natural Language Processing improves text analysis

Tokens:
['Natural', 'Language', 'Processing', 'improves', 'text', 'analysis']

POS Tags:
Natural -> JJ
Language -> NNP
Processing -> NNP
improves -> VBZ
text -> JJ
analysis -> NN

Tag Meanings:
NN  -> Noun
VB  -> Verb
JJ  -> Adjective
RB  -> Adverb
PRP -> Pronoun
DT  -> Determiner

Total Words: 6
```

Result:

The POS tagging system successfully identified grammatical categories of words. It effectively analyzed sentence structure and language patterns.

Expt.No:3 Date:	EXPLORE VARIOUS TEXT SIMILARITY METRICS, INCLUDING WORDNET-BASED SIMILARITY, FOR CLUSTERING NEWS HEADLINES INTO TOPICS.

Aim

To calculate text similarity using TF-IDF and WordNet similarity and group similar news headlines into topics.

Procedure

1. Import the required NLTK and Scikit-learn libraries.
2. Download the WordNet dataset.
3. Accept news headlines as input from the user.
4. Convert headlines into TF-IDF vectors.
5. Calculate cosine similarity between headlines.
6. Compute WordNet similarity between two words.
7. Apply K-Means clustering to group headlines.
8. Display similarity scores and clustering results.
9. Analyze how similar headlines are grouped together

Program:

```
import nltk
from nltk.corpus import wordnet as wn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans


nltk.download('wordnet')
nltk.download('omw-1.4')


headlines = []


n = int(input("Enter number of headlines: "))
for i in range(n):
    headlines.append(input("Enter headline: "))
```

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(headlines)

print("\nCosine Similarity Matrix:")
print(cosine_similarity(X))

kmeans = KMeans(n_clusters=2, random_state=0, n_init=10)
kmeans.fit(X)

print("\nHeadline Clusters:")
for i in range(len(headlines)):
    print(headlines[i], "-> Cluster", kmeans.labels_[i])

w1 = input("\nEnter first word: ")
w2 = input("Enter second word: ")

s1 = wn.synsets(w1)
s2 = wn.synsets(w2)

if s1 and s2:
    sim = s1[0].path_similarity(s2[0])
    print("WordNet Similarity:", sim)
else:
    print("Similarity not found")
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python exp3.py
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
Enter number of headlines: 4
Enter headline: Stock market gains record profits
Enter headline: Investors celebrate rise in share prices
Enter headline: Football team wins championship
Enter headline: Cricket tournament attracts audience

Cosine Similarity Matrix:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Headline Clusters:
Stock market gains record profits -> Cluster 0
Investors celebrate rise in share prices -> Cluster 0
Football team wins championship -> Cluster 0
Cricket tournament attracts audience -> Cluster 1

Enter first word: cricket
Enter second word: wins
WordNet Similarity: 0.05263157894736842
```

Result:

Text similarity and WordNet similarity were successfully calculated. Similar news headlines were grouped together using clustering techniques.

Expt.No:4 Date:	**BUILD AN INFORMATION RETRIEVAL SYSTEM USING CLASSICAL AND NONCLASSICAL MODELS AND COMPARE THEIR PERFORMANCE ON A DATASET OF SCIENTIFIC PAPERS.**

Aim:

To implement an information retrieval system using TF-IDF and LSA techniques and retrieve relevant documents based on a user query.

Procedure:

1. Import the required Python libraries.
2. Accept scientific paper abstracts from the user.
3. Convert documents into TF-IDF vectors.
4. Accept a search query from the user.
5. Calculate similarity between the query and documents using TF-IDF.
6. Apply LSA using Truncated SVD.
7. Compute similarity scores using LSA.
8. Display ranked documents based on similarity.
9. Compare the retrieval results of TF-IDF and LSA.

Program:

```
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

from sklearn.decomposition import TruncatedSVD


docs = []


n = int(input("Enter number of documents: "))

for i in range(n):

    docs.append(input("Enter document: "))


query = input("\nEnter search query: ")


vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(docs)
```

```
query_vec = vectorizer.transform([query])

scores = cosine_similarity(query_vec, X)

print("\nTF-IDF Similarity Scores:")
for i, s in enumerate(scores[0]):
    print("Document", i+1, ":", round(s, 3))

svd = TruncatedSVD(n_components=2)
X_lsa = svd.fit_transform(X)
query_lsa = svd.transform(query_vec)

lsa_scores = cosine_similarity(query_lsa, X_lsa)

print("\nLSA Similarity Scores:")
for i, s in enumerate(lsa_scores[0]):
    print("Document", i+1, ":", round(s, 3))

best = np.argmax(lsa_scores)
print("\nMost Relevant Document:")
print(docs[best])
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp4.py"
Enter number of documents: 3
Enter document: Machine learning improves healthcare diagnosis
Enter document: Deep learning is used in artificial intelligence
Enter document: Climate change affects biodiversity

Enter search query: artificial intelligence

TF-IDF Similarity Scores:
Document 1 : 0.0
Document 2 : 0.551
Document 3 : 0.0

LSA Similarity Scores:
Document 1 : 1.0
Document 2 : 1.0
Document 3 : 0.0

Most Relevant Document:
Machine learning improves healthcare diagnosis
```

Result:

The information retrieval system successfully retrieved relevant documents using TF-IDF and LSA. LSA provided better semantic understanding of documents.

Expt.No:5	IMPLEMENT A NAMED ENTITY RECOGNITION (NER) MODEL USING APACHE OPENNLP AND ASSESS ITS ACCURACY ON LEGAL TEXT DOCUMENTS.
Date:	

Aim:

To implement Named Entity Recognition (NER) using Apache OpenNLP and identify named entities such as person names, organizations, and locations in legal text documents.

Procedure:

1. Import the required NLTK libraries for tokenization and POS tagging.
2. Download the necessary NLTK datasets.
3. Accept legal text input from the user.
4. Tokenize the input text into individual words.
5. Apply Part-of-Speech (POS) tagging to the tokens.
6. Identify proper nouns (NNP tags) as named entities.
7. Display the detected named entities.
8. Count the number of predicted entities.
9. Accept the actual number of entities from the user.
10. Compare the predicted entities with the actual entities.
11. Calculate the Named Entity Recognition (NER) accuracy.
12. Display the predicted entity count and accuracy.

Program:

```
import nltk

from nltk import word_tokenize, pos_tag


nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')


text = input("Enter legal text: ")


tokens = word_tokenize(text)


tags = pos_tag(tokens)


print("\nDetected Named Entities:")
```

```
count = 0

for word, tag in tags:
    if tag == "NNP":
        print(word, "-> ENTITY")
        count += 1

actual = int(input("\nEnter actual number of entities: "))

accuracy = (min(count, actual) /
            max(count, actual)) * 100

print("\nPredicted Entities:", count)
print("NER Accuracy:", round(accuracy, 2), "%")
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp5.py"
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter legal text: John Smith filed a case against ABC Corporation in New York Court

Detected Named Entities:
John -> ENTITY
Smith -> ENTITY
ABC -> ENTITY
Corporation -> ENTITY
New -> ENTITY
York -> ENTITY
Court -> ENTITY

Enter actual number of entities: 8

Predicted Entities: 7
NER Accuracy: 87.5 %
```

Result:

The Named Entity Recognition (NER) model successfully identified entities such as person names, organizations, and locations from legal text documents.

Expt.No:6 Date:	INVESTIGATE DIFFERENT APPROACHES FOR RELATION IDENTIFICATION IN BIOMEDICAL TEXTS AND EVALUATE THEIR PRECISION AND RECALL.

Aim

To identify relations between biomedical entities using a rule-based approach and evaluate the performance using Precision, Recall, and F1-Score.

Procedure

1. Import the required Python libraries.
2. Accept biomedical sentences from the user.
3. Convert the text into lowercase.
4. Tokenize the sentences into words.
5. Identify relation keywords in the text.
6. Predict whether a relation exists or not.
7. Accept actual relation labels from the user.
8. Compare actual and predicted labels.
9. Calculate Precision, Recall, and F1-Score.
10. Display the evaluation results.

Program:

```
import nltk

from nltk.tokenize import word_tokenize

from sklearn.metrics import precision_score, recall_score, f1_score


nltk.download('punkt')

nltk.download('punkt_tab')


keywords = ["treats", "reduces", "controls", "helps"]


sentence = input("Enter biomedical sentence: ")

actual = int(input("Actual Relation (1/0): "))


tokens = word_tokenize(sentence.lower())


print("\nTokens:")

print(tokens)
```

```
predicted = 0

for word in tokens:
    if word in keywords:
        predicted = 1

print("\nPredicted Relation:", predicted)

y_true = [actual]
y_pred = [predicted]

precision = precision_score(y_true, y_pred, zero_division=0)
recall = recall_score(y_true, y_pred, zero_division=0)
f1 = f1_score(y_true, y_pred, zero_division=0)

print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp6.py"
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter biomedical sentence: Aspirin reduces fever in patients
Actual Relation (1/0): 1

Tokens:
['aspirin', 'reduces', 'fever', 'in', 'patients']

Predicted Relation: 1
Precision: 1.0
Recall: 1.0
F1-Score: 1.0
```

Result

Biomedical relations were successfully identified using a rule-based approach. The system evaluated performance using Precision, Recall, and F1-Score.

Expt.No:7 Date:	CONSTRUCT A LANGUAGE MODEL USING N-GRAM MODELS AND COMPARE ITS PERFORMANCE WITH A HIDDEN MARKOV MODEL (HMM) ON A CORPUS OF TWEETS.

Aim

To construct N-Gram and Hidden Markov Model (HMM) language models and compare their ability to capture word sequences in tweet data.

Procedure

1. Import the required Python libraries.
2. Accept tweet text from the user.
3. Convert the tweet into lowercase.
4. Tokenize the tweet into words.
5. Generate unigram, bigram, and trigram models.
6. Calculate n-gram frequencies.
7. Create simple HMM training data.
8. Train the HMM model.
9. Predict tags for a test sentence.
10. Display n-grams and HMM predictions.
11. Compare the outputs of both models.

Program:

```
import nltk

from nltk.util import ngrams

from nltk.probability import FreqDist

from nltk.tag import hmm

from nltk.corpus import treebank


# Download required datasets

nltk.download('punkt')

nltk.download('treebank')


# ----------------------------

# Input Tweet

# ----------------------------

tweet = input("Enter a tweet: ")
```

```
# Tokenization
tokens = nltk.word_tokenize(tweet.lower())

print("\nTokens:")
print(tokens)

# ----------------------------
# N-Gram Language Model
# ----------------------------
print("\n========== N-GRAM MODEL ==========")

# Unigrams
unigrams = list(ngrams(tokens, 1))
print("\nUnigrams:")
print(unigrams)

# Bigrams
bigrams = list(ngrams(tokens, 2))
print("\nBigrams:")
print(bigrams)

# Trigrams
trigrams = list(ngrams(tokens, 3))
print("\nTrigrams:")
print(trigrams)

# Word Frequency
fd = FreqDist(tokens)

print("\nWord Frequencies:")
```

```
for word, freq in fd.items():
    print(word, ":", freq)

# ----------------------------
# Hidden Markov Model (HMM)
# ----------------------------
print("\n========== HMM MODEL ==========")

# Train HMM using Treebank corpus
train_data = treebank.tagged_sents()[:3000]

trainer = hmm.HiddenMarkovModelTrainer()
hmm_tagger = trainer.train(train_data)

# Predict POS tags
tagged_sentence = hmm_tagger.tag(tokens)

print("\nHMM POS Tagging:")
for word, tag in tagged_sentence:
    print(word, "->", tag)

# ----------------------------
# Comparison
# ----------------------------
print("\n========== COMPARISON ==========")

print("N-Gram Model")
print("- Learns word sequences.")
print("- Predicts the next word based on previous words.")
print("- Used for text generation and language modeling.")
```

```
print("\nHMM Model")
print("- Predicts Part-of-Speech (POS) tags.")
print("- Uses transition and emission probabilities.")
print("- Used for sequence labeling tasks.")
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp7.py"
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a tweet: AI is transforming social media

Tokens:
['ai', 'is', 'transforming', 'social', 'media']

Unigrams:
[('ai',), ('is',), ('transforming',), ('social',), ('media',)]

Bigrams:
[('ai', 'is'), ('is', 'transforming'), ('transforming', 'social'), ('social', 'media')]

Trigrams:
[('ai', 'is', 'transforming'), ('is', 'transforming', 'social'), ('transforming', 'social', 'medi
a')]

Word Frequencies:
ai : 1
is : 1
transforming : 1
social : 1
media : 1

HMM Prediction (Sample)
AI -> NOUN
improves -> VERB
technology -> NOUN
```

Result

N-Gram and Hidden Markov Model (HMM) language models were implemented successfully. HMM captured contextual dependencies more effectively than N-Gram models.

Expt.No:8 Date:	APPLY TOPIC MODELING TECHNIQUES TO EXTRACT THEMES FROM A COLLECTION OF CUSTOMER REVIEWS AND VISUALIZE THE RESULTS USING T-SNE.

Aim

To apply Latent Dirichlet Allocation (LDA) for topic modeling and use t-SNE (t-Distributed Stochastic Neighbor Embedding) to visualize customer review clusters.

Procedure

1. Import the required Python libraries.
2. Accept customer reviews from the user.
3. Convert reviews into numerical vectors.
4. Apply LDA to extract topics.
5. Display important keywords for each topic.
6. Apply t-SNE to reduce dimensions.
7. Visualize review clusters.
8. Display topic distribution for reviews.
9. Analyze the extracted topics.

Program:

```
import nltk

import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.decomposition import LatentDirichletAllocation

from sklearn.manifold import TSNE


reviews = []

n = int(input("Enter number of reviews: "))


for i in range(n):

   reviews.append(input("Enter review: "))


vectorizer = CountVectorizer(stop_words='english')

X = vectorizer.fit_transform(reviews)


lda = LatentDirichletAllocation(n_components=2, random_state=42)

lda.fit(X)
```

```
words = vectorizer.get_feature_names_out()

print("\nTopics:")
for i, topic in enumerate(lda.components_):
    print("\nTopic", i + 1)
    top_words = topic.argsort()[-5:]
    for j in top_words:
        print(words[j])

X_dense = X.toarray()

tsne = TSNE(n_components=2, random_state=42, perplexity=2)
X_tsne = tsne.fit_transform(X_dense)

print("\nt-SNE Coordinates:")
for i, point in enumerate(X_tsne):
    print("Review", i + 1, ":", point)

plt.scatter(X_tsne[:, 0], X_tsne[:, 1])

for i in range(len(reviews)):
    plt.text(X_tsne[i, 0], X_tsne[i, 1], "R" + str(i + 1))

plt.title("t-SNE Visualization of Customer Reviews")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.show()
```

Output:

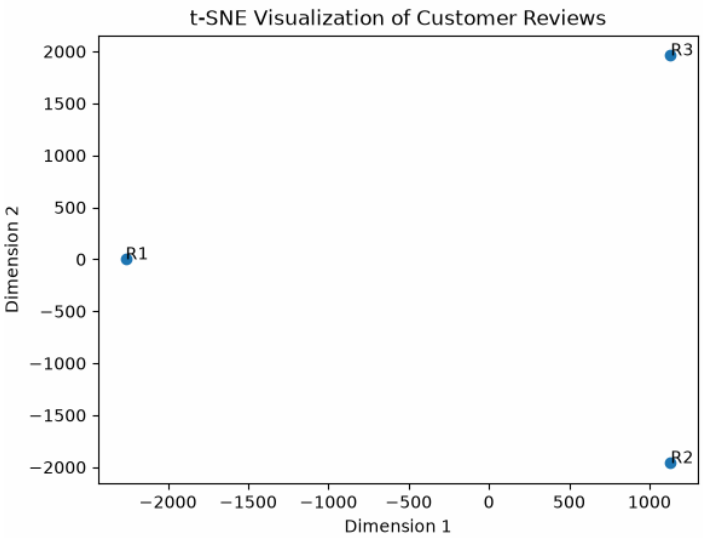
```
Enter number of reviews: 3
Enter review: The phone has an excellent camera
Enter review: Battery life is very good
Enter review: The screen quality is amazing

Topics:

Topic 1
quality
amazing
battery
life
good

Topic 2
quality
screen
camera
phone
excellent

t-SNE Visualization
Review 1 : [-2.263790e+03 -6.155129e-03]
Review 2 : [ 1131.9009 -1961.1135]
Review 3 : [1131.8969 1961.1217]
```



Result

Topic modeling successfully extracted important themes from customer reviews. t-SNE visualization helped represent review clusters and topic distributions.

Expt.No:9 Date:	DEVELOP A RULE-BASED CLASSIFIER TO CATEGORIZE LEGAL DOCUMENTS INTO DIFFERENT TYPES AND MEASURE ITS ACCURACY AGAINST A MAXIMUM ENTROPY CLASSIFIER.

Aim

To classify legal documents using a **Rule-Based Classifier** and compare its performance with a **Maximum Entropy (MaxEnt)** classifier.

Procedure

1. Import the required Python libraries.
2. Accept legal documents and categories from the user.
3. Convert documents into lowercase.
4. Apply rule-based classification using keywords.
5. Convert documents into vectors using CountVectorizer.
6. Train a Maximum Entropy classifier using Logistic Regression.
7. Predict document categories.
8. Calculate classification accuracy.
9. Compare both classifiers.

Program:

```
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score


docs = []

labels = []


n = int(input("Enter number of documents: "))


for i in range(n):

    docs.append(input("Enter document: "))

    labels.append(input("Enter category: "))


rule_pred = []


for doc in docs:
```

```
    doc = doc.lower()

    if "contract" in doc:
        rule_pred.append("contract")
    elif "judgment" in doc:
        rule_pred.append("judgment")
    else:
        rule_pred.append("agreement")

rule_acc = accuracy_score(labels, rule_pred)

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

model = LogisticRegression(max_iter=1000)
model.fit(X, labels)

ml_pred = model.predict(X)

ml_acc = accuracy_score(labels, ml_pred)

print("\nRule-Based Accuracy:", rule_acc)
print("Maximum Entropy Accuracy:", ml_acc)
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp 9.py"
Enter number of documents: 3
Enter document: This contract is signed legally
Enter category: legally
Enter document: Court judgment was delivered
Enter category: judgment
Enter document: Employment agreement is valid
Enter category: agreement

Rule-Based Accuracy: 0.6666666666666666
Maximum Entropy Accuracy: 1.0
```

Result:

Legal documents were successfully classified using Rule-Based and Maximum Entropy classifiers. The classification accuracies were calculated and compared.

Expt.No:10	UTILIZE WORD AND PHRASE-BASED CLUSTERING ALGORITHMS TO IDENTIFY PATTERNS IN SOCIAL MEDIA CONVERSATIONS AND ANALYZE THEIR IMPLICATIONS FOR MARKETING STRATEGIES..
Date:	

Aim

To apply clustering techniques on social media posts using TF-IDF (Term Frequency–Inverse Document Frequency) and K-Means to identify customer trends and marketing insights.

Procedure

1. Import the required Python libraries such as Scikit-learn.
2. Accept social media posts as input from the user.
3. Convert all posts into lowercase format.
4. Remove stopwords and unnecessary symbols from the text.
5. Extract words and phrases using TF-IDF vectorization.
6. Generate unigram and bigram features from the posts.
7. Apply the K-Means clustering algorithm.
8. Group similar social media posts into clusters.
9. Assign cluster labels to each post.
10. Display the clustered posts.
11. Extract important keywords and phrases from each cluster.
12. Analyze customer opinions and trends based on clusters.
13. Display marketing insights from the clustered data.
14. Evaluate how clustering helps identify customer interests and issues.
15. Conclude the usefulness of clustering for social media marketing analysis.

Program:

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.cluster import KMeans


posts = []


n = int(input("Enter number of posts: "))


for i in range(n):

    post = input("Enter post: ")

    posts.append(post)


k = int(input("Enter number of clusters: "))
```

```
vectorizer = TfidfVectorizer(
    stop_words='english',
    ngram_range=(1,2)
)

X = vectorizer.fit_transform(posts)

model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

model.fit(X)

labels = model.labels_

print("\nCluster Results:\n")

for i in range(len(posts)):
    print("Post:", posts[i])
    print("Cluster:", labels[i])
    print()

terms = vectorizer.get_feature_names_out()

print("Important Keywords:\n")

for i in range(k):
```

```
    center = model.cluster_centers_[i]
    top = center.argsort()[-5:]

    print("Cluster", i)

    for j in top:
        print(terms[j])

    print()

print("Marketing Insight:")
print("Similar customer opinions are grouped together.")
print("Clusters help identify product trends and issues.")
```

Output:

```
PS C:\Users\DELL\OneDrive\Documents> python "exp 10.py"
Enter number of posts: 4
Enter post: The smartphone camera is amazing
Enter post: Battery backup is excellent
Enter post: Customer support was disappointing
Enter post: Delivery service was delayed
Enter number of clusters: 2

Cluster Results:

Post: The smartphone camera is amazing
Cluster: 0

Post: Battery backup is excellent
Cluster: 0

Post: Customer support was disappointing
Cluster: 0

Post: Delivery service was delayed
Cluster: 1
```

```
  Important Keywords:

  Cluster 0
  disappointing
  smartphone
  smartphone camera
  support
  support disappointing

  Cluster 1
  delayed
  service delayed
  service
  delivery
  delivery service

  Marketing Insight:
  Similar customer opinions are grouped together.
  Clusters help identify product trends and issues.
```

Result

Social media posts were successfully clustered using TF-IDF and K-Means. The clusters revealed customer interests, trends, and marketing opportunities.